LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5

CONNECT TO THE INTERNET

Oleh:

Amandha Citra Mustika NIM. 2410817320004

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amandha Citra Mustika
NIM : 2410817320004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
b. Gunakan KotlinX Serialization sebagai library JSON.
c. Gunakan library seperti Coil atau Glide untuk image loading.
d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started
e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

MainActivity.kt Compose

Table 1. Source Code MainActivity.kt Compose

```
1  package com.example.mymovieappm5
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.activity.enableEdgeToEdge
7  import androidx.compose.runtime.*
8  import androidx.lifecycle.viewmodel.compose.viewModel
```

```
9  import androidx.navigation.compose.NavHost
10 import androidx.navigation.compose.composable
11 import androidx.navigation.compose.rememberNavController
12 import com.example.mymovieappm5.data.local.MovieDatabase
13 import
   com.example.mymovieappm5.data.repository.MovieRepository
14 import com.example.mymovieappm5.domain.model.Movie
15 import com.example.mymovieappm5.ui.screen.DetailScreen
16 import com.example.mymovieappm5.ui.screen.HomeScreen
17 import com.example.mymovieappm5.ui.screen.LanguageScreen
18 import com.example.mymovieappm5.ui.theme.MyMovieAppM5Theme
19 import com.example.mymovieappm5.ui.viewmodel.MovieViewModel
20 import
   com.example.mymovieappm5.ui.viewmodel.MovieViewModelFactory
21
22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25         enableEdgeToEdge()
26
27         val database = MovieDatabase.getInstance(this)
28         val repository = MovieRepository(database)
29         val factory = MovieViewModelFactory(repository)
30
31         setContent {
32             MyMovieAppM5Theme {
33                 val navController = rememberNavController()
34                 val viewModel: MovieViewModel =
   viewModel(factory = factory)
35                 var selectedMovie by remember {
   mutableStateOf<Movie?>(null) }
36
37                 NavHost(navController = navController,
   startDestination = "language") {
38                     composable("language") {
39                         LanguageScreen(navController =
   navController)
40                     }
41                     composable("home") {
42                         if (selectedMovie != null) {
43                             DetailScreen(
44                                 movie = selectedMovie!!,
45                                 onBackClick = {
   selectedMovie = null }
46                             )
47                         } else {
48                             HomeScreen(
```

```
49                                      viewModel = viewModel,
50                                      onLanguageClick = {
51                                  navController.navigate("language")
    {
52                                              launchSingleTop =
    true
53                                          }
54                                      },
55                                      onMovieClick = { movie ->
    selectedMovie = movie }
56                                  )
57                              }
58                          }
59                      }
60                  }
61              }
62          }
63      }
```

Movie.kt Compose

Table 2. Source Code Movie.kt Compose

```
1   package com.example.mymovieappm5.domain.model
2
3   data class Movie(
4       val id: Int,
5       val title: String,
6       val overview: String,
7       val posterPath: String?,
8       val backdropPath: String?,
9       val releaseDate: String,
10      val voteAverage: Double,
11      val voteCount: Int,
12      val popularity: Double
13  )
```

MovieDao.kt Compose

Table 3. Source Code MovieDao.kt Compose

```
1   package com.example.mymovieappm5.data.local
2
3   import androidx.room.Dao
4   import androidx.room.Insert
5   import androidx.room.OnConflictStrategy
```

```
6  import androidx.room.Query
7  import kotlinx.coroutines.flow.Flow
8
9  @Dao
10 interface MovieDao {
11     @Query("SELECT * FROM movies WHERE category =
   :category")
12     fun getMoviesByCategory(category: String):
   Flow<List<MovieEntity>>
13
14     @Insert(onConflict = OnConflictStrategy.REPLACE)
15     suspend fun insertMovies(movies: List<MovieEntity>)
16
17     @Query("DELETE FROM movies WHERE category = :category")
18     suspend fun deleteMoviesByCategory(category: String)
19
20     @Query("SELECT * FROM movies WHERE id = :id")
21     suspend fun getMovieById(id: Int): MovieEntity?
22
23     @Query("SELECT cachedAt FROM movies WHERE category =
   :category LIMIT 1")
24     suspend fun getCacheTime(category: String): Long?
25 }
```

MovieDatabase.kt Compose

Table 4. Source Code MovieDatabase.kt Compose

```
1  package com.example.mymovieappm5.data.local
2
3  import android.content.Context
4  import androidx.room.Database
5  import androidx.room.Room
6  import androidx.room.RoomDatabase
7
8  @Database(entities = [MovieEntity::class], version = 1,
   exportSchema = false)
9  abstract class MovieDatabase : RoomDatabase() {
10     abstract fun movieDao(): MovieDao
11
12     companion object {
13         @Volatile
14         private var INSTANCE: MovieDatabase? = null
15
16         fun getInstance(context: Context): MovieDatabase {
17             return INSTANCE ?: synchronized(this) {
18                 Room.databaseBuilder(
```

```
                    context.applicationContext,
                    MovieDatabase::class.java,
                    "movie_database"
                ).build().also { INSTANCE = it }
            }
        }
    }
}
```

MovieEntity.kt Compose

Table 5. Source Code MovieEntity.kt Compose

```
package com.example.mymovieappm5.data.local

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "movies")
data class MovieEntity(
    @PrimaryKey val id: Int,
    val title: String,
    val overview: String,
    val posterPath: String?,
    val backdropPath: String?,
    val releaseDate: String,
    val voteAverage: Double,
    val voteCount: Int,
    val popularity: Double,
    val category: String,
    val cachedAt: Long = System.currentTimeMillis()
)
```

MovieRepository.kt Compose

Table 6. Source Code MovieRepository.kt Compose

```
package com.example.mymovieappm5.data.repository

import com.example.mymovieappm5.data.local.MovieDatabase
import com.example.mymovieappm5.data.local.MovieEntity
import
com.example.mymovieappm5.data.remote.RetrofitInstance
import com.example.mymovieappm5.domain.model.Movie
import com.example.mymovieappm5.util.Constants
import com.example.mymovieappm5.util.Resource
```

```
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow

class MovieRepository(private val database: MovieDatabase) {

    fun getPopularMovies(): Flow<Resource<List<Movie>>> = flow {
        emit(Resource.Loading())
        try {
            val cacheTime = database.movieDao().getCacheTime(Constants.CATEGORY_POPULAR)
            val isCacheValid = cacheTime != null &&
                    System.currentTimeMillis() - cacheTime < Constants.CACHE_DURATION

            if (isCacheValid) {

database.movieDao().getMoviesByCategory(Constants.CATEGORY_POPULAR)
                    .collect { entities ->
                        emit(Resource.Success(entities.map { it.toMovie() }))
                    }
            } else {
                val response = RetrofitInstance.api.getPopularMovies(Constants.API_KEY)
                val entities = response.results.map {
                    MovieEntity(
                        id = it.id,
                        title = it.title,
                        overview = it.overview,
                        posterPath = it.posterPath,
                        backdropPath = it.backdropPath,
                        releaseDate = it.releaseDate,
                        voteAverage = it.voteAverage,
                        voteCount = it.voteCount,
                        popularity = it.popularity,
                        category = Constants.CATEGORY_POPULAR
                    )
                }

database.movieDao().deleteMoviesByCategory(Constants.CATEGORY_POPULAR)
```

```
46                  database.movieDao().insertMovies(entities)
47  database.movieDao().getMoviesByCategory(Constants.CATEGORY
    _POPULAR)
48                      .collect { cached ->
49                          emit(Resource.Success(cached.map {
    it.toMovie() }))
50                      }
51              }
52          } catch (e: Exception) {
53              emit(Resource.Error(e.message ?: "Terjadi
    kesalahan"))
54          }
55      }
56
57      fun getNowPlayingMovies(): Flow<Resource<List<Movie>>>
    = flow {
58          emit(Resource.Loading())
59          try {
60              val cacheTime =
    database.movieDao().getCacheTime(Constants.CATEGORY_NOW_PL
    AYING)
61              val isCacheValid = cacheTime != null &&
62                      System.currentTimeMillis() - cacheTime
    < Constants.CACHE_DURATION
63              if (isCacheValid) {
64  database.movieDao().getMoviesByCategory(Constants.CATEGORY
    _NOW_PLAYING)
65                      .collect { entities ->
66                          emit(Resource.Success(entities.map
    { it.toMovie() }))
67                      }
68              } else {
69                  val response =
    RetrofitInstance.api.getNowPlayingMovies(Constants.API_KEY
    )
70                  val entities = response.results.map {
71                      MovieEntity(
72                          id = it.id,
73                          title = it.title,
74                          overview = it.overview,
75                          posterPath = it.posterPath,
76                          backdropPath = it.backdropPath,
77                          releaseDate = it.releaseDate,
78                          voteAverage = it.voteAverage,
79                          voteCount = it.voteCount,
80                          popularity = it.popularity,
81
```

```
                            category =
82  Constants.CATEGORY_NOW_PLAYING
83                          )
84                  }
85
    database.movieDao().deleteMoviesByCategory(Constants.CATEG
86  ORY_NOW_PLAYING)
87                  database.movieDao().insertMovies(entities)
    database.movieDao().getMoviesByCategory(Constants.CATEGORY
88  _NOW_PLAYING)
89                          .collect { cached ->
                                emit(Resource.Success(cached.map {
90  it.toMovie() }))
91                          }
92              }
93          } catch (e: Exception) {
                emit(Resource.Error(e.message ?: "Terjadi
94  kesalahan"))
95          }
96      }
97
        suspend fun searchMovies(query: String):
98  Resource<List<Movie>> {
99          return try {
                val response =
    RetrofitInstance.api.searchMovies(Constants.API_KEY,
100 query)
101             Resource.Success(response.results.map { it ->
102                 Movie(
103                     id = it.id,
104                     title = it.title,
105                     overview = it.overview,
106                     posterPath = it.posterPath,
107                     backdropPath = it.backdropPath,
108                     releaseDate = it.releaseDate,
109                     voteAverage = it.voteAverage,
110                     voteCount = it.voteCount,
111                     popularity = it.popularity
112                 )
113             })
114         } catch (e: Exception) {
                Resource.Error(e.message ?: "Terjadi
115 kesalahan")
116         }
117     }
118
119     private fun MovieEntity.toMovie() = Movie(
```

```
120             id = id,
121             title = title,
122             overview = overview,
123             posterPath = posterPath,
124             backdropPath = backdropPath,
125             releaseDate = releaseDate,
126             voteAverage = voteAverage,
127             voteCount = voteCount,
128             popularity = popularity
129         )
    }
```

MovieResponse.kt Compose

Table 7. Source Code MovieResponse.kt Compose

```
1   package com.example.mymovieappm5.data.remote
2
3   import kotlinx.serialization.SerialName
4   import kotlinx.serialization.Serializable
5
6   @Serializable
7   data class MovieResponse(
8       @SerialName("page") val page: Int,
9       @SerialName("results") val results: List<MovieDto>,
10      @SerialName("total_pages") val totalPages: Int,
11      @SerialName("total_results") val totalResults: Int
12  )
13
14  @Serializable
15  data class MovieDto(
16      @SerialName("id") val id: Int,
17      @SerialName("title") val title: String,
18      @SerialName("overview") val overview: String,
19      @SerialName("poster_path") val posterPath: String? =
    null,
20      @SerialName("backdrop_path") val backdropPath: String?
    = null,
21      @SerialName("release_date") val releaseDate: String =
    "",
22      @SerialName("vote_average") val voteAverage: Double =
    0.0,
23      @SerialName("vote_count") val voteCount: Int = 0,
24      @SerialName("popularity") val popularity: Double = 0.0
25  )
```

MovieViewModel.kt Compose

Table 8. Source Code MovieViewModel.kt Compose

```
1   package com.example.mymovieappm5.ui.viewmodel
2
3   import androidx.lifecycle.ViewModel
4   import androidx.lifecycle.viewModelScope
5   import
    com.example.mymovieappm5.data.repository.MovieRepository
6   import com.example.mymovieappm5.domain.model.Movie
7   import com.example.mymovieappm5.util.Resource
8   import kotlinx.coroutines.flow.MutableStateFlow
9   import kotlinx.coroutines.flow.StateFlow
10  import kotlinx.coroutines.launch
11
12  class MovieViewModel(private val repository:
13  MovieRepository) : ViewModel() {
14
15      private val _popularMovies =
    MutableStateFlow<Resource<List<Movie>>>(Resource.Loading())
16      val popularMovies: StateFlow<Resource<List<Movie>>> =
    _popularMovies
17
18      private val _nowPlayingMovies =
    MutableStateFlow<Resource<List<Movie>>>(Resource.Loading())
19      val nowPlayingMovies: StateFlow<Resource<List<Movie>>>
    = _nowPlayingMovies
20
21      private val _searchResults =
    MutableStateFlow<Resource<List<Movie>>>(Resource.Loading())
22      val searchResults: StateFlow<Resource<List<Movie>>> =
    _searchResults
23      private val _searchQuery = MutableStateFlow("")
24      val searchQuery: StateFlow<String> = _searchQuery
25
26      init {
27          getPopularMovies()
28          getNowPlayingMovies()
29      }
30
31      fun getPopularMovies() {
32          viewModelScope.launch {
33              repository.getPopularMovies().collect {
34                  _popularMovies.value = it
35              }
36          }
```

```
37      }
38
39      fun getNowPlayingMovies() {
40          viewModelScope.launch {
41              repository.getNowPlayingMovies().collect {
42                  _nowPlayingMovies.value = it
43              }
44          }
45      }
46
47      fun searchMovies(query: String) {
48          _searchQuery.value = query
49          viewModelScope.launch {
50              _searchResults.value = Resource.Loading()
51              _searchResults.value =
52  repository.searchMovies(query)
            }
53      }
54  }
```

MovieViewModelFactory.kt Compose

Table 9. Source Code MovieViewModelFactory.kt Compose

```
1   package com.example.mymovieappm5.ui.viewmodel
2
3   import androidx.lifecycle.ViewModel
4   import androidx.lifecycle.ViewModelProvider
    import
5   com.example.mymovieappm5.data.repository.MovieRepository

6   class MovieViewModelFactory(private val repository:
7   MovieRepository) : ViewModelProvider.Factory {
        override fun <T : ViewModel> create(modelClass:
8   Class<T>): T {
            if
9   (modelClass.isAssignableFrom(MovieViewModel::class.java)) {
                @Suppress("UNCHECKED_CAST")
10              return MovieViewModel(repository) as T
11          }
12          throw IllegalArgumentException("Unknown ViewModel
13  class")
        }
14  }
15
```

RetrofitInstance.kt Compose

Table 10. Source Code RetrofitInstance.kt Compose

```
1  package com.example.mymovieappm5.data.remote
2
3  import
   com.jakewharton.retrofit2.converter.kotlinx.serialization.a
   sConverterFactory
4  import kotlinx.serialization.json.Json
5  import okhttp3.MediaType.Companion.toMediaType
6  import okhttp3.OkHttpClient
7  import okhttp3.logging.HttpLoggingInterceptor
8  import retrofit2.Retrofit
9
10 object RetrofitInstance {
11     private const val BASE_URL =
   "https://api.themoviedb.org/3/"
12
13     private val json = Json {
14         ignoreUnknownKeys = true
15     }
16
17     private val loggingInterceptor =
   HttpLoggingInterceptor().apply {
18         level = HttpLoggingInterceptor.Level.BODY
19     }
20
21     private val client = OkHttpClient.Builder()
22         .addInterceptor(loggingInterceptor)
23         .build()
24
25     val api: ApiService by lazy {
26         Retrofit.Builder()
27             .baseUrl(BASE_URL)
28             .client(client)
29 .addConverterFactory(json.asConverterFactory("application/j
   son".toMediaType()))
30             .build()
31             .create(ApiService::class.java)
32     }
33 }
```

ApiService.kt Compose

Table 11. Source Code ApiService.kt Compose

```
1   package com.example.mymovieappm5.data.remote
2
3   import retrofit2.http.GET
4   import retrofit2.http.Path
5   import retrofit2.http.Query
6
7   interface ApiService {
8       @GET("movie/popular")
9       suspend fun getPopularMovies(
10          @Query("api_key") apiKey: String,
11          @Query("page") page: Int = 1
12      ): MovieResponse
13
14      @GET("movie/now_playing")
15      suspend fun getNowPlayingMovies(
16          @Query("api_key") apiKey: String,
17          @Query("page") page: Int = 1
18      ): MovieResponse
19
20      @GET("search/movie")
21      suspend fun searchMovies(
22          @Query("api_key") apiKey: String,
23          @Query("query") query: String,
24          @Query("page") page: Int = 1
25      ): MovieResponse
26  }
```

Constants.kt Compose

Table 12. Source Code Constants.kt Compose

```
1   package com.example.mymovieappm5.util
2
3   object Constants {
4       const val API_KEY = "b61a592770cb876e92e77cdc349dfc59"
5       const val IMAGE_BASE_URL =
    "https://image.tmdb.org/t/p/w500"
6       const val CACHE_DURATION = 60 * 60 * 1000L
7       const val CATEGORY_POPULAR = "popular"
8       const val CATEGORY_NOW_PLAYING = "now_playing"
9   }
```

Resource.kt Compose

Table 13. Source Code Resource.kt Compose

```
1   package com.example.songpediacompose
2
3   import androidx.compose.runtime.Composable
4   import androidx.compose.ui.platform.LocalContext
5   import androidx.lifecycle.viewmodel.compose.viewModel
6   import androidx.navigation.compose.NavHost
7   import androidx.navigation.compose.composable
8   import androidx.navigation.compose.rememberNavController
9   import
    com.example.songpediacompose.viewmodel.MovieViewModel
10  import
    com.example.songpediacompose.viewmodel.MovieViewModelFactor
    y
11
12  @Composable
13  fun NavGraph() {
14      val navController = rememberNavController()
15      val context = LocalContext.current
16      val viewModel: MovieViewModel = viewModel(
17          factory =
    MovieViewModelFactory(context.applicationContext as
    android.app.Application, "remote")
18      )
19
20      NavHost(navController = navController, startDestination
    = "home") {
21          composable("home") {
22              HomeScreen(
23                  viewModel = viewModel,
24                  onDetailClick = { movie ->
25                      viewModel.selectMovie(movie)
26                      navController.navigate("detail")
27                  },
28                  onLanguageClick = {
29                      navController.navigate("language")
30                  }
31              )
32          }
33          composable("detail") {
34              DetailScreen(
35                  viewModel = viewModel,
36                  onBack = {
37                      navController.popBackStack()
```

```
38                  }
39              )
40          }
41          composable("language") {
42              LanguageScreen(
43                  onBack = {
44                      navController.popBackStack()
45                  }
46              )
47          }
48      }
49  }
```

HomeScreen.kt Compose

Table 14. Source Code HomeScreen.kt Compose

```
1   package com.example.mymovieappm5.ui.screen
2
3   import androidx.compose.foundation.background
4   import androidx.compose.foundation.layout.*
5   import androidx.compose.foundation.lazy.LazyColumn
6   import androidx.compose.foundation.lazy.LazyRow
7   import androidx.compose.foundation.lazy.items
8   import
    androidx.compose.foundation.shape.RoundedCornerShape
9   import androidx.compose.material.icons.Icons
10  import androidx.compose.material.icons.filled.Language
11  import androidx.compose.material.icons.filled.Star
12  import androidx.compose.material3.*
13  import androidx.compose.runtime.*
14  import androidx.compose.ui.Alignment
15  import androidx.compose.ui.Modifier
16  import androidx.compose.ui.draw.clip
17  import androidx.compose.ui.graphics.Color
18  import androidx.compose.ui.layout.ContentScale
19  import androidx.compose.ui.text.font.FontWeight
20  import androidx.compose.ui.text.style.TextOverflow
21  import androidx.compose.ui.unit.dp
22  import androidx.compose.ui.platform.LocalContext
23  import androidx.compose.ui.unit.sp
24  import android.content.Intent
25  import android.net.Uri
26  import coil.compose.AsyncImage
27  import com.example.mymovieappm5.domain.model.Movie
```

```
28  import
    com.example.mymovieappm5.ui.viewmodel.MovieViewModel
29  import com.example.mymovieappm5.util.Constants
30  import com.example.mymovieappm5.util.Resource
31
32  val ComicBackground = Color(0xFFF2F0F8)
33  val ComicSurface = Color(0xFFFFFFFF)
34  val ComicPurple = Color(0xFF7C3AED)
35  val ComicPurpleLight = Color(0xFFEDE9FE)
36  val ComicTextPrimary = Color(0xFF1A1A2E)
37  val ComicTextSecondary = Color(0xFF6B7280)
38  val ComicStar = Color(0xFFFBBF24)
39
40  @OptIn(ExperimentalMaterial3Api::class)
41  @Composable
42  fun HomeScreen(
43      viewModel: MovieViewModel,
44      onLanguageClick: () -> Unit,
45      onMovieClick: (Movie) -> Unit
46  ) {
47      val popularMovies by
    viewModel.popularMovies.collectAsState()
48      val nowPlayingMovies by
    viewModel.nowPlayingMovies.collectAsState()
49      val searchResults by
    viewModel.searchResults.collectAsState()
50      val searchQuery by
    viewModel.searchQuery.collectAsState()
51      var isSearching by remember { mutableStateOf(false) }
52
53      Box(
54          modifier = Modifier
55              .fillMaxSize()
56              .background(ComicBackground)
57      ) {
58          LazyColumn(modifier = Modifier.fillMaxSize()) {
59              item {
60                  Row(
61                      modifier = Modifier
62                          .fillMaxWidth()
63                          .padding(horizontal = 16.dp,
    vertical = 16.dp),
64                      horizontalArrangement =
    Arrangement.SpaceBetween,
65                      verticalAlignment =
    Alignment.CenterVertically
66                  ) {
```

```
67                      Text(
68                          text = "Movie App",
69                          color = ComicTextPrimary,
70                          fontSize = 22.sp,
71                          fontWeight = FontWeight.Bold
72                      )
73                      IconButton(onClick = {
    onLanguageClick() }) {
74                          Icon(
75                              Icons.Default.Language,
76                              contentDescription =
    "Language",
77                              tint = ComicPurple
78                          )
79                      }
80                  }
81              }
82
83              if (isSearching) {
84                  item {
85                      OutlinedTextField(
86                          value = searchQuery,
87                          onValueChange = {
    viewModel.searchMovies(it) },
88                          modifier = Modifier
89                              .fillMaxWidth()
90                              .padding(horizontal = 16.dp,
    vertical = 8.dp),
91                          placeholder = { Text("Cari
    film...", color = ComicTextSecondary) },
92                          singleLine = true,
93                          colors =
    OutlinedTextFieldDefaults.colors(
94                              focusedBorderColor =
    ComicPurple,
95                              unfocusedBorderColor =
    ComicPurpleLight,
96                              focusedTextColor =
    ComicTextPrimary,
97                              unfocusedTextColor =
    ComicTextPrimary,
98                              cursorColor = ComicPurple
99                          )
100                     )
101                 }
102                 item {
103                     when (searchResults) {
```

```
104                         is Resource.Loading -> {
105                             if (searchQuery.isNotEmpty())
106 {
107
108 Box(Modifier.fillMaxWidth(), contentAlignment =
    Alignment.Center) {
109 CircularProgressIndicator(color = ComicPurple)
110                                 }
111                             }
112                         }
113                         is Resource.Success -> {
114                             val movies = (searchResults as
    Resource.Success).data
115                             Column {
116                                 movies.forEach { movie ->
117                                     AllMoviesCard(movie =
118 movie, onMovieClick = { onMovieClick(movie) })
                                 }
119                             }
120                         }
121                         is Resource.Error -> {
122                             Text(
123                                 text = (searchResults as
124 Resource.Error).message,
                                 modifier =
125 Modifier.padding(16.dp),
                                 color = Color.Red
126                             )
127                         }
128                     }
129                 }
130             } else {
131                 item {
132                     Text(
133                         text = "Featured Movies",
134                         color = ComicTextPrimary,
135                         fontSize = 18.sp,
136                         fontWeight = FontWeight.Bold,
137                         modifier =
138 Modifier.padding(horizontal = 16.dp, vertical = 4.dp)
                     )
139                 }
140                 item {
141                     when (nowPlayingMovies) {
142                         is Resource.Loading -> {
143                             Box(
144
```

```
145                                     modifier =
    Modifier.fillMaxWidth().height(260.dp),
146                                     contentAlignment =
    Alignment.Center
147                                 ) {
148 CircularProgressIndicator(color = ComicPurple) }
149                             }
150                             is Resource.Success -> {
151                                 val movies = (nowPlayingMovies
    as Resource.Success).data
152                                 LazyRow(
153                                     contentPadding =
    PaddingValues(horizontal = 16.dp, vertical = 8.dp),
154                                     horizontalArrangement =
    Arrangement.spacedBy(12.dp)
155                                 ) {
156                                     items(movies) { movie ->
157                                         FeaturedMovieCard(
158                                             movie = movie,
159                                             onDetailClick = {
    onMovieClick(movie) }
160                                         )
161                                     }
162                                 }
163                             }
164                             is Resource.Error -> {
165                                 Text(
166                                     text = (nowPlayingMovies
    as Resource.Error).message,
167                                     modifier =
    Modifier.padding(16.dp),
168                                     color = Color.Red
169                                 )
170                             }
171                         }
172                 }
173
174                 item {
175                     Text(
176                         text = "All Movies",
177                         color = ComicTextPrimary,
178                         fontSize = 18.sp,
179                         fontWeight = FontWeight.Bold,
180                         modifier = Modifier.padding(start
    = 16.dp, end = 16.dp, top = 12.dp, bottom = 4.dp)
181                     )
182                 }
```

```
183                 when (popularMovies) {
184                     is Resource.Loading -> {
185                         item {
186                             Box(Modifier.fillMaxWidth(),
187 contentAlignment = Alignment.Center) {
188 CircularProgressIndicator(color = ComicPurple)
189                             }
190                         }
191                     }
192                     is Resource.Success -> {
193                         val movies = (popularMovies as
    Resource.Success).data
194                         items(movies) { movie ->
195                             AllMoviesCard(
196                                 movie = movie,
197                                 onMovieClick = {
198 onMovieClick(movie) }
199                             )
200                         }
201                     }
202                     is Resource.Error -> {
203                         item {
204                             Text(
205                                 text = (popularMovies as
    Resource.Error).message,
206                                 modifier =
    Modifier.padding(16.dp),
207                                 color = Color.Red
208                             )
209                         }
210                     }
211                 }
212                 item { Spacer(modifier =
    Modifier.height(24.dp)) }
213             }
214         }
215     }
216 }
217
218 @Composable
219 fun FeaturedMovieCard(movie: Movie, onDetailClick: () ->
    Unit) {
220     Card(
221         modifier = Modifier.width(160.dp),
222         shape = RoundedCornerShape(16.dp),
223
```

```
        colors = CardDefaults.cardColors(containerColor =
224 ComicSurface),
        elevation =
225 CardDefaults.cardElevation(defaultElevation = 4.dp)
226     ) {
227         Column {
228             AsyncImage(
                    model = Constants.IMAGE_BASE_URL +
229 movie.posterPath,
230                 contentDescription = movie.title,
231                 modifier = Modifier
232                     .fillMaxWidth()
233                     .height(200.dp),
234                 contentScale = ContentScale.Crop
235             )
                Column(modifier = Modifier.padding(horizontal
236 = 10.dp, vertical = 6.dp)) {
237                 Text(
238                     text = movie.title,
239                     color = ComicTextPrimary,
240                     fontWeight = FontWeight.SemiBold,
241                     fontSize = 13.sp,
242                     maxLines = 1,
243                     overflow = TextOverflow.Ellipsis
244                 )
245                 Text(
246                     text = movie.releaseDate.take(4),
247                     color = ComicTextSecondary,
248                     fontSize = 11.sp
249                 )
250                 Spacer(modifier = Modifier.height(6.dp))
251                 Button(
252                     onClick = onDetailClick,
253                     modifier = Modifier
254                         .fillMaxWidth()
255                         .height(34.dp),
                        colors =
256 ButtonDefaults.buttonColors(containerColor = ComicPurple),
257                     shape = RoundedCornerShape(10.dp),
258                     contentPadding = PaddingValues(0.dp)
259                 ) {
                        Text("Detail", color = Color.White,
260 fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
261                 }
262                 Spacer(modifier = Modifier.height(8.dp))
263             }
264         }
```

```
    }
}

@Composable
fun AllMoviesCard(movie: Movie, onMovieClick: () -> Unit)
{
    val context = LocalContext.current

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 6.dp),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor =
ComicSurface),
        elevation =
CardDefaults.cardElevation(defaultElevation = 3.dp)
    ) {
        Row(modifier = Modifier.padding(10.dp)) {
            AsyncImage(
                model = Constants.IMAGE_BASE_URL +
movie.posterPath,
                contentDescription = movie.title,
                modifier = Modifier
                    .width(75.dp)
                    .height(105.dp)
                    .clip(RoundedCornerShape(10.dp)),
                contentScale = ContentScale.Crop
            )
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement =
Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.Top
                ) {
                    Text(
                        text = movie.title,
                        color = ComicTextPrimary,
                        fontWeight = FontWeight.Bold,
                        fontSize = 15.sp,
                        maxLines = 2,
                        overflow = TextOverflow.Ellipsis,
                        modifier = Modifier.weight(1f)
                    )
                    Text(
```

```
308                             text = movie.releaseDate.take(4),
309                             color = ComicTextSecondary,
310                             fontSize = 12.sp,
                                modifier = Modifier.padding(start
311 = 4.dp)
312                         )
313                     }
314                     Spacer(modifier = Modifier.height(4.dp))
                        Row(verticalAlignment =
315 Alignment.CenterVertically) {
316                         Icon(
317                             imageVector = Icons.Default.Star,
318                             contentDescription = null,
319                             tint = ComicStar,
320                             modifier = Modifier.size(14.dp)
321                         )
322                         Spacer(modifier =
    Modifier.width(3.dp))
323                         Text(
324                             text = String.format("%.1f",
    movie.voteAverage),
325                             color = ComicTextSecondary,
326                             fontSize = 12.sp
327                         )
328                     }
329                     Spacer(modifier = Modifier.height(10.dp))
330                     Row(horizontalArrangement =
    Arrangement.spacedBy(8.dp)) {
331                         Button(
332                             onClick = {
333                                 val url =
    "https://www.themoviedb.org/movie/${movie.id}"
334                                 val intent =
    Intent(Intent.ACTION_VIEW, Uri.parse(url))
335                                 context.startActivity(intent)
                                },
336                             colors =
    ButtonDefaults.buttonColors(containerColor = ComicPurple),
337                             shape = RoundedCornerShape(8.dp),
338                             contentPadding =
    PaddingValues(horizontal = 18.dp, vertical = 0.dp),
339                             modifier = Modifier.height(32.dp)
340                         ) {
341                             Text("TMDB", color = Color.White,
    fontSize = 12.sp, fontWeight = FontWeight.SemiBold)
342                         }
343                         OutlinedButton(
```

```
344                         onClick = onMovieClick,
345                         shape = RoundedCornerShape(8.dp),
346                         colors =
    ButtonDefaults.outlinedButtonColors(contentColor =
    ComicPurple),
347                         border =
    androidx.compose.foundation.BorderStroke(1.5.dp,
    ComicPurple),
348                         contentPadding =
    PaddingValues(horizontal = 18.dp, vertical = 0.dp),
349                         modifier = Modifier.height(32.dp)
350                     ) {
351                         Text("Detail", fontSize = 12.sp,
352 fontWeight = FontWeight.SemiBold)
354                     }
355                 }
356             }
357         }
358     }
359 }
```

DetailScreen.kt Compose

Table 15. Source Code DetailScreen.kt Compose

```
1   package com.example.mymovieappm5.ui.screen
2
3   import androidx.compose.foundation.layout.*
4   import androidx.compose.foundation.rememberScrollState
5   import androidx.compose.foundation.verticalScroll
6   import androidx.compose.material.icons.Icons
7   import androidx.compose.material.icons.filled.ArrowBack
8   import androidx.compose.material3.*
9   import androidx.compose.runtime.*
10  import androidx.compose.ui.Modifier
11  import androidx.compose.ui.layout.ContentScale
12  import androidx.compose.ui.text.font.FontWeight
13  import androidx.compose.ui.unit.dp
14  import androidx.compose.ui.unit.sp
15  import coil.compose.AsyncImage
16  import com.example.mymovieappm5.domain.model.Movie
17  import com.example.mymovieappm5.util.Constants
18
19  @OptIn(ExperimentalMaterial3Api::class)
20  @Composable
21  fun DetailScreen(
22      movie: Movie,
```

```
23      onBackClick: () -> Unit
24  ) {
25      Scaffold(
26          topBar = {
27              TopAppBar(
28                  title = { Text(movie.title, maxLines = 1)
29  },
30                  navigationIcon = {
31                      IconButton(onClick = onBackClick) {
32                          Icon(Icons.Default.ArrowBack,
    contentDescription = "Back")
33                      }
34                  }
35              )
36          }
37      ) { padding ->
38          Column(
39              modifier = Modifier
40                  .fillMaxSize()
41                  .padding(padding)
42                  .verticalScroll(rememberScrollState())
43          ) {
44              AsyncImage(
45                  model = Constants.IMAGE_BASE_URL +
    movie.backdropPath,
46                  contentDescription = movie.title,
47                  modifier = Modifier
48                      .fillMaxWidth()
49                      .height(350.dp),
50                  contentScale = ContentScale.Crop
51              )
52              Column(modifier = Modifier.padding(16.dp)) {
53                  Row(
54                      modifier = Modifier.fillMaxWidth(),
55                      horizontalArrangement =
    Arrangement.SpaceBetween
56                  ) {
57                      Text(
58                          text = movie.title,
59                          fontWeight = FontWeight.Bold,
60                          fontSize = 24.sp,
61                          modifier = Modifier.weight(1f)
62                      )
63                      Text(
64                          text = movie.releaseDate.take(4),
65                          fontSize = 16.sp,
66                          fontWeight = FontWeight.Bold
```

```
67                      )
68                  }
69                  Spacer(modifier = Modifier.height(8.dp))
70                  Text(
71                      text = "⭐ ${String.format("%.1f",
    movie.voteAverage)} (${movie.voteCount} votes)",
72                      fontSize = 14.sp
73                  )
74                  Text(
75                      text = "Popularitas:
    ${String.format("%.1f", movie.popularity)}",
76                      fontSize = 14.sp
77                  )
78                  Spacer(modifier = Modifier.height(12.dp))
79                  Text(
80                      text = "Sinopsis",
81                      fontWeight = FontWeight.Bold,
82                      fontSize = 16.sp
83                  )
84                  Spacer(modifier = Modifier.height(4.dp))
85                  Text(
86                      text = movie.overview,
87                      fontSize = 14.sp
88                  )
89              }
90          }
91      }
92  }
```

LanguageScreen.kt Compose

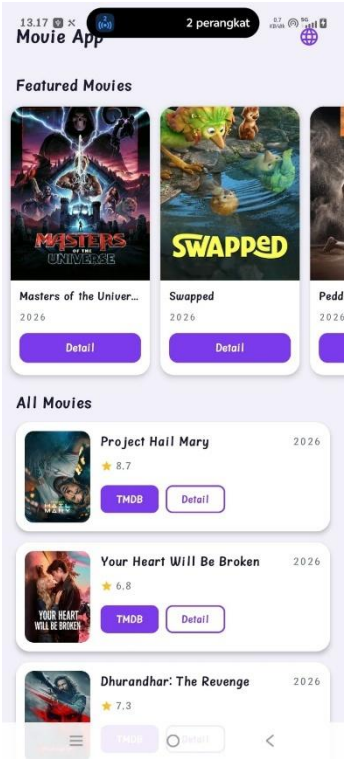
Table 16. Source Code LanguageScreen.kt Compose

```
1   package com.example.mymovieappm5.ui.screen
2
3   import android.content.res.Configuration
4   import androidx.compose.foundation.layout.*
5   import
    androidx.compose.foundation.shape.RoundedCornerShape
6   import androidx.compose.material3.Button
7   import androidx.compose.material3.ButtonDefaults
8   import androidx.compose.material3.Text
9   import androidx.compose.runtime.Composable
10  import androidx.compose.ui.Alignment
11  import androidx.compose.ui.Modifier
12  import androidx.compose.ui.graphics.Color
13  import androidx.compose.ui.platform.LocalContext
```

```
14  import androidx.compose.ui.text.font.FontWeight
15  import androidx.compose.ui.unit.dp
16  import androidx.compose.ui.unit.sp
17  import androidx.navigation.NavController
18  import java.util.Locale
19
20  @Composable
21  fun LanguageScreen(navController: NavController) {
22      val context = LocalContext.current
23
24      Column(
25          modifier = Modifier
26              .fillMaxSize()
27              .padding(32.dp),
28          verticalArrangement = Arrangement.Center,
29          horizontalAlignment = Alignment.CenterHorizontally
30      ) {
31          Text(
32              text = "Choose Language",
33              fontWeight = FontWeight.Bold,
34              fontSize = 22.sp,
35              modifier = Modifier.padding(bottom = 32.dp)
36          )
37
38          Button(
39              onClick = {
40                  val locale = Locale("en")
41                  Locale.setDefault(locale)
42                  val config =
    Configuration(context.resources.configuration)
43                  config.setLocale(locale)
44
45  context.resources.updateConfiguration(config,
46  context.resources.displayMetrics)
47                  navController.navigate("home") {
48                      popUpTo("language") { inclusive = true
49  }
50                      launchSingleTop = true
51                  }
52              },
53              modifier = Modifier
54                  .fillMaxWidth()
55                  .height(52.dp),
56              colors =
    ButtonDefaults.buttonColors(containerColor = ComicPurple),
57              shape = RoundedCornerShape(12.dp)
58          ) {
```

```
59              Text("English", color = Color.White,
    fontWeight = FontWeight.SemiBold)
60          }
61
62          Spacer(modifier = Modifier.height(16.dp))
63
64          Button(
65              onClick = {
66                  val locale = Locale("in")
67                  Locale.setDefault(locale)
68                  val config =
    Configuration(context.resources.configuration)
69                  config.setLocale(locale)
70
71  context.resources.updateConfiguration(config,
72  context.resources.displayMetrics)
73                  navController.navigate("home") {
74                      popUpTo("language") { inclusive = true
75  }
76                      launchSingleTop = true
77                  }
78              },
79              modifier = Modifier
80                  .fillMaxWidth()
81                  .height(52.dp),
82              colors =
    ButtonDefaults.buttonColors(containerColor = ComicPurple),
83              shape = RoundedCornerShape(12.dp)
84          ) {
85              Text("Bahasa Indonesia", color = Color.White,
    fontWeight = FontWeight.SemiBold)
86          }
87      }
88  }
```

B. Output Program



Gambar 1. Screenshot Hasil Tampilan List Jawaban Soal 1

Gambar 2. Screenshot Hasil Tampilan Detail Jawaban Soal 1

Gambar 3. Screenshot Hasil Tampilan Bahasa Jawaban Soal 1

C. Pembahasan

MainActivity.kt

File 'MainActivity.kt' mewarisi 'ComponentActivity' sebagai entry point aplikasi. Di dalam 'onCreate()', database diinisialisasi melalui 'MovieDatabase.getInstance()', lalu 'repository' dan 'factory' dibuat untuk menyuplai 'MovieViewModel'. Navigasi menggunakan 'NavHost' dengan dua route 'language' sebagai start destination dan 'home' yang menampilkan 'HomeScreen' atau 'DetailScreen' tergantung apakah 'selectedMovie' bernilai null atau tidak. State 'selectedMovie' dikelola menggunakan 'remember' dan 'mutableStateOf' untuk menentukan tampilan aktif tanpa berpindah route.

Constants.kt

File 'Constants.kt' berisi objek singleton yang menyimpan seluruh konstanta global aplikasi. 'API_KEY' menyimpan kunci autentikasi untuk mengakses TMDB API, 'IMAGE_BASE_URL' menyimpan base URL untuk memuat poster film, dan

'CACHE_DURATION' menetapkan durasi cache selama 1 jam dalam satuan milidetik. Selain itu terdapat dua konstanta kategori film yaitu 'CATEGORY_POPULAR' dan 'CATEGORY_NOW_PLAYING' yang digunakan sebagai parameter request ke API.

MovieEntity.kt

File 'MovieEntity.kt' adalah data class yang merepresentasikan struktur tabel 'movies' di database Room, ditandai dengan anotasi '@Entity'. '@PrimaryKey' diletakkan pada properti 'id' sebagai identifikasi unik setiap baris. Dibanding domain model 'Movie', class ini memiliki dua properti tambahan — 'category' untuk membedakan film berdasarkan kategori ('popular' atau 'now_playing'), dan 'cachedAt' yang secara otomatis diisi dengan waktu saat data disimpan menggunakan 'System.currentTimeMillis()' untuk keperluan validasi cache di repository.

HomeScreen.kt

File 'HomeScreen.kt' menampilkan dua daftar film yang datanya diambil dari 'viewModel.popularMovies' dan 'viewModel.nowPlayingMovies' menggunakan 'collectAsState()'. Keduanya bertipe 'Resource' sehingga UI menangani tiga kondisi: loading dengan 'CircularProgressIndicator', error dengan pesan teks, dan success dengan menampilkan data. Section 'Featured Movies' menampilkan film now playing dalam 'LazyRow', sementara 'All Movies' menampilkan film populer dalam 'LazyColumn' menggunakan composable 'AllMoviesCard'. Tombol TMDB pada setiap kartu membuka halaman film langsung di browser menggunakan 'Intent.ACTION_VIEW', sedangkan tombol Detail mengarahkan ke 'DetailScreen' melalui callback 'onMovieClick'.

Movie.kt

File 'Movie.kt' mendefinisikan data class yang merepresentasikan model domain film. Class ini memiliki sembilan properti — 'id' sebagai identifikasi unik film, 'title' dan 'overview' sebagai informasi utama, 'posterPath' dan 'backdropPath' bertipe nullable untuk URL gambar, 'releaseDate' untuk tanggal rilis, serta 'voteAverage', 'voteCount', dan 'popularity' untuk data penilaian film dari TMDB.

DetailScreen.kt

File 'DetailScreen.kt' menampilkan detail lengkap sebuah film yang menerima parameter 'movie' dan 'onBackClick'. Layar menggunakan 'Scaffold' dengan 'TopAppBar' yang menampilkan judul film dan tombol back untuk kembali ke 'HomeScreen'. Konten utama ditampilkan dalam 'Column' yang bisa di-scroll menggunakan 'verticalScroll', diawali dengan gambar backdrop menggunakan 'AsyncImage' dengan URL dari 'Constants.IMAGE_BASE_URL'. Di bawahnya ditampilkan judul, tahun rilis, rating, jumlah votes, popularitas, dan sinopsis film dari properti 'movie.overview'.

LanguageScreen.kt

File 'LanguageScreen.kt' menampilkan halaman pemilihan bahasa aplikasi yang muncul pertama kali saat aplikasi dibuka. Terdapat dua tombol pilihan 'English' dan 'Bahasa Indonesia' yang masing-masing menerapkan 'Locale' sesuai pilihan menggunakan 'Configuration' dan 'updateConfiguration'. Setelah bahasa diterapkan, navigasi langsung berpindah ke route 'home' dengan 'popUpTo("language") { inclusive = true }' agar halaman language tidak bisa diakses kembali melalui tombol back.

MovieResponse.kt

File 'MovieResponse.kt' mendefinisikan dua data class untuk menangani respons dari TMDB API. 'MovieResponse' merepresentasikan struktur utama respons yang berisi 'page', 'totalPages', 'totalResults', dan 'results' berupa list 'MovieDto'. 'MovieDto' merepresentasikan data mentah satu film dari API dengan anotasi '@SerialName' untuk memetakan nama field JSON ke properti Kotlin. Beberapa properti seperti 'posterPath', 'backdropPath', dan 'releaseDate' memiliki nilai default untuk mengantisipasi field yang mungkin tidak tersedia dalam respons API.

ApiService.kt

File 'ApiService.kt' mendefinisikan interface Retrofit yang berisi tiga fungsi suspend untuk mengakses TMDB API. 'getPopularMovies' mengambil daftar film populer melalui endpoint 'movie/popular', 'getNowPlayingMovies' mengambil film yang sedang tayang melalui 'movie/now_playing', dan 'searchMovies' melakukan pencarian film berdasarkan query melalui

'search/movie'. Ketiga fungsi menerima 'api_key' dan 'page' sebagai query parameter, serta mengembalikan 'MovieResponse' sebagai hasil respons dari API.

Resource.kt

File 'Resource.kt' mendefinisikan sealed class yang digunakan sebagai wrapper state untuk hasil operasi data di seluruh aplikasi. Terdapat tiga subclass — 'Success' membawa data hasil operasi bertipe generic 'T', 'Error' membawa pesan kesalahan bertipe String, dan 'Loading' sebagai penanda bahwa operasi sedang berlangsung. Dengan menggunakan sealed class, UI dapat menangani ketiga kondisi tersebut secara exhaustive menggunakan 'when' expression tanpa memerlukan kondisi else.

RetrofitInstance.kt

File 'RetrofitInstance.kt' mendefinisikan objek singleton untuk konfigurasi Retrofit. 'BASE_URL' mengarah ke endpoint utama TMDB API, sementara 'Json { ignoreUnknownKeys = true }' dikonfigurasi agar parsing tidak gagal ketika ada field tambahan dari respons API yang tidak terdapat di data class. 'OkHttpClient' ditambahkan 'HttpLoggingInterceptor' dengan level 'BODY' untuk mencatat detail request dan response saat debugging. Instance 'api' dibuat secara lazy menggunakan 'Retrofit.Builder' dengan converter factory dari kotlinx serialization untuk mengubah respons JSON menjadi objek Kotlin.

MovieDao.kt

File 'MovieDao.kt' adalah interface DAO yang mendefinisikan operasi database untuk tabel 'movies'. 'getMoviesByCategory' mengembalikan 'Flow<List<MovieEntity>>' agar UI bisa reaktif terhadap perubahan data secara real-time. 'insertMovies' menyimpan list film dengan strategi 'REPLACE' sehingga data lama otomatis diganti jika terdapat konflik. 'deleteMoviesByCategory' menghapus seluruh film berdasarkan kategori, 'getMovieById' mengambil satu film berdasarkan id, dan 'getCacheTime' mengambil waktu cache terakhir berdasarkan kategori untuk keperluan validasi apakah data perlu diperbarui dari API atau tidak.

MovieDatabase.kt

File 'MovieDatabase.kt' adalah class abstract yang mewarisi 'RoomDatabase' dan berfungsi sebagai entry point utama database lokal aplikasi. Anotasi '@Database' mendefinisikan 'MovieEntity' sebagai satu-satunya entitas dengan versi database 1. Instance database dibuat menggunakan pola singleton dengan anotasi '@Volatile' untuk memastikan nilai 'INSTANCE' selalu konsisten antar thread, dan blok 'synchronized' untuk mencegah pembuatan instance ganda saat diakses secara bersamaan. Database dibangun menggunakan 'Room.databaseBuilder' dengan nama file 'movie_database' dan hanya menggunakan 'applicationContext' untuk menghindari memory leak.

MovieRepository.kt

File 'MovieRepository.kt' adalah class yang menjadi jembatan antara data layer (API dan database lokal) dengan domain layer. Fungsi 'getPopularMovies' dan 'getNowPlayingMovies' mengembalikan 'Flow<Resource<List<Movie>>>' dan menerapkan strategi caching sebelum mengambil data dari API, repository mengecek 'getCacheTime' terlebih dahulu dan membandingkannya dengan 'Constants.CACHE_DURATION'. Jika cache masih valid, data diambil langsung dari Room tanpa memanggil API. Jika cache sudah kedaluwarsa, data baru diambil dari API, disimpan ke database menggunakan 'insertMovies', lalu dikembalikan ke UI. Fungsi 'searchMovies' tidak menggunakan cache karena bersifat dinamis langsung memanggil API dan mengembalikan 'Resource<List<Movie>>' secara langsung. Extension function 'toMovie' bertugas mengkonversi 'MovieEntity' dari database menjadi domain model 'Movie'.

MovieViewModel.kt

File 'MovieViewModel.kt' adalah class yang mewarisi 'ViewModel' dan bertugas mengelola state UI serta berkomunikasi dengan 'MovieRepository'. Terdapat empat 'MutableStateFlow' yang masing-masing menyimpan state untuk 'popularMovies', 'nowPlayingMovies', 'searchResults', dan 'searchQuery', yang semuanya diekspos sebagai 'StateFlow' read-only ke UI. Blok 'init' langsung memanggil 'getPopularMovies' dan 'getNowPlayingMovies' saat ViewModel pertama kali dibuat. Fungsi 'searchMovies' memperbarui 'searchQuery', lalu memanggil 'repository.searchMovies' di dalam 'viewModelScope.launch' dan

hasilnya disimpan ke '_searchResults'. Seluruh operasi coroutine dijalankan dalam 'viewModelScope' agar otomatis dibatalkan saat ViewModel dihancurkan.

MovieViewModelFactory.kt

File 'MovieViewModelFactory.kt' adalah class yang mewarisi 'ViewModelProvider.Factory' dan bertugas membuat instance 'MovieViewModel' dengan menyuntikkan 'MovieRepository' sebagai dependensi. Fungsi 'create' mengecek apakah class yang diminta adalah 'MovieViewModel' menggunakan 'isAssignableFrom', lalu mengembalikan instance-nya dengan repository yang sudah disiapkan. Jika class yang diminta bukan 'MovieViewModel', fungsi ini melempar 'IllegalArgumentException'. Factory ini diperlukan karena 'MovieViewModel' membutuhkan parameter constructor sehingga tidak bisa dibuat langsung oleh sistem Android tanpa factory.

TAUTAN GIT

Berikut adalah tautan untuk *source code* yang telah dibuat.

https://github.com/zvyaa/MODUL-MOBILE